

CS2 Week 11

Practical MPC



Kissan Varatharajan - kissanv.github.io

Exercise Classes

- Shared exercise with Haixiao and Kissan
- Theory recap
- Notes available on Kissan's website: <https://kissanv.github.io>
& on Haixiao's Polybox: <https://polybox.ethz.ch/index.php/s/AReA9YBeJQKxBDn>
- Quizzes, Examples & Old exam questions



Kissan's Website



Haixiao's Polybox

Course Schedule

Subject	Week
Intro – Discrete Time Systems	1
Diagonalization, Modal Coordinates, Contr/Obs.	2
State Feedback, Pole Placement, LQR	3
State Estimation, Luenberger Obs., Separation Principle, LQG	4
DOFC with integral action, Intro to MIMO systems	5
Norms	6
Nonlinear Systems, Lyapunov, Feedback Linearization	7
break	-
Discrete-time Unconstrained Optimal Control	8
Convex Optimization	9
Model Predictive Control	10
Practical Model Predictive Control	11
Robust Model Predictive Control	12
Implementation Methods	13

Last week we focused on mathematical guarantees MPC, we used:

- **Invariant Sets:** We used these to guarantee feasibility. If controller finds safe path now, it will never run out of options in the future.
- **Lyapunov Functions:** We used these to guarantee stability. By making MPC cost function behave like a Lyapunov function $\rightarrow$ cost always drops, forcing it to origin.

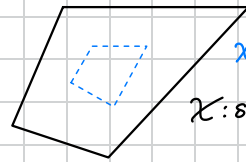
This week, we look at:

- how we can achieve tracking (not forcing state to zero, but track r)
- rejecting disturbances. Constant disturbances cause offset from origin, we want to remove offset, s.t. system converges to desired set point.
- enlargening the feasible set. Constraints restrict set of states, for which opt. problem is feasible. We want it as big as possible.

Tracking

we care about piecewise constant reference

Goal: track given reference r , s.t. $z(k) = Hx(k) \rightarrow r \in \mathbb{R}^n$ as $k \rightarrow \infty$



X_w : feasible set

X : state constraints

The standard MPC problem regulates system state to origin:

$$J^*(x(k)) = \underset{U_k}{\operatorname{argmin}} \left(\ell(x_N) + \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i}) \right)$$

e.g. quadratic cost: $x^T Q x + u^T R u$

sub. to $x_k = x(k)$

$$x_{k+i+1} = A x_{k+i} + B u_{k+i}$$

$$x_{k+i} \in X$$

$$u_{k+i} \in U$$

$$U_k = \{u_k, u_{k+1}, \dots, u_{k+N-1}\}$$

measurement
system model
state constraints
input constraints
opt. variables

$\Rightarrow$ How can we modify this MPC problem to achieve tracking?

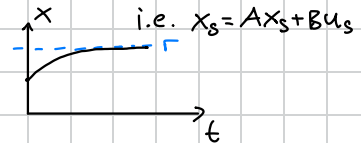
We need to change the following:

- adapt MPC cost
- adapt terminal set

Steady-State target problem

1. Reference r is achieved by target state x_s if $z_s = Hx_s = r$ linear comb. of state, maps $x_s \rightarrow z_s$

2. Target state should be a steady-state $\rightarrow$ there exists an input that keeps system there,



Summarized:

- $x_s = Ax_s + Bu_s$

rewrite in
matrix form $\rightarrow$

- $Hx_s = r$

$$\underbrace{\begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix}}_{(n_x+n_r) \times (n_x+n_u)} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$

Further important:

1. If there are any **constraints** present, (x_s, u_s) has to satisfy them.
2. If the problem has multiple solutions u_s , we compute the 'cheapest' (x_s, u_s) corresponding to r .

translated:

$$\begin{array}{ll} \min & u_s^T R u_s \\ \text{subi. to} & \begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix} \\ & x_s \in \mathcal{X}, u_s \in \mathcal{U} \end{array}$$

If on the other hand there is no solution, we compute 'closest' reachable set point to r .

translated:

$$\begin{array}{ll} \min & (Hx_s - r)^T Q_s (Hx_s - r) \\ \text{subi. to} & x_s = Ax_s + Bu_s \\ & x_s \in \mathcal{X}, u_s \in \mathcal{U} \end{array}$$

MPC for Reference Tracking

Now that we know, how to calculate (x_s, u_s) to reach r , we want to control the system to this desired steady-state condition, yielding $z(k) \rightarrow r$.

So we want to penalize deviations from (x_s, u_s) . We use the 2-norm for this.

The MPC is designed as:

$$\min_U \|z_N - Hx_s\|_{P_z}^2 + \sum_{i=0}^{N-1} \|z_i - Hx_s\|_{Q_z}^2 + \|u_i - u_s\|_R^2$$

Notation: $(u_i - u_s)^T R (u_i - u_s)$

subj. to [model, constraints]

$x_0 = x(k)$

We have now formulated a tracking problem, but we are more familiar with regulation problems. Thus we want to rewrite the problem above to a regulation problem.

- We do this by defining the following deviation variables:

$$\Delta x = x - x_s$$

$$\begin{aligned}\Delta x_{k+1} &= x_{k+1} - x_s \\ &= Ax_k + Bu_k - (Ax_s + Bu_s) \\ &= A\Delta x + B\Delta u\end{aligned}$$

$$\Delta u = u - u_s$$

- We also rewrite the constraints for deviation variables:

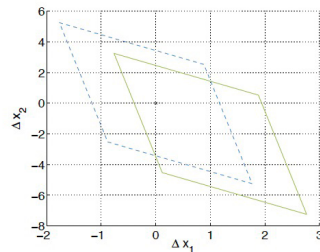
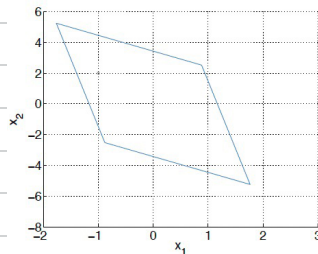
- $G_x x_s$ on both sides

$$G_x x \leq h_x \Rightarrow G_x \Delta x \leq h_x - G_x x_s$$

- $G_u u_s$ on both sides

$$G_u u \leq h_u \Rightarrow G_u \Delta u \leq h_u - G_u u_s$$

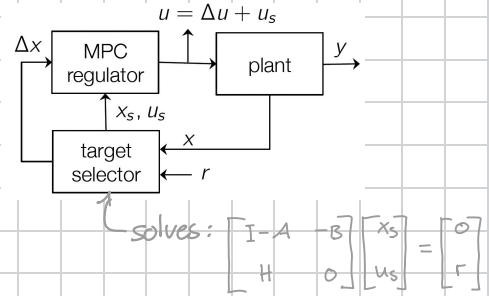
Graphically this means a shift:



We now have all building blocks we need. We can return to our MPC formulation...

everything put together:

$$\begin{aligned} \min \quad & \sum_{i=0}^{N-1} \Delta x_i^T Q \Delta x_i + \Delta u_i^T R \Delta u_i + V_f(\Delta x_N) \\ \text{s.t.} \quad & \Delta x_0 = \Delta x(k) \\ & \Delta x_{i+1} = A \Delta x_i + B \Delta u_i \\ & G_x \Delta x_i \leq h_x - G_x x_s \\ & G_u \Delta u_i \leq h_u - G_u u_s \\ & \Delta x_N \in \mathcal{X}_f \end{aligned}$$



Here we find the optimal sequence of Δu^* and after apply the input $u_0^* = \Delta u_0^* + u_s$ to the system.

Does this delta formulation preserve the guarantees we proved for regulating to zero?

Assume target is feasible with $x_s \in \mathcal{X}$, $u_s \in \mathcal{U}$ and choose terminal constraint $\mathcal{X}_f = \{0\}$ (i.e. $\Delta x_N = 0$, $x_N = x_s$). Then CL system converges to target reference. i.e. $x \rightarrow x_s$ and therefore $z(k) = Hx(k) \rightarrow r$ for $k \rightarrow \infty$

Quick remark:

So far we assumed we can measure state and goal is $z(k) = Hx(k) \rightarrow r$ for $k \rightarrow \infty$

In practice we often can't measure these, but we can measure output $y(k) = Cx(k)$

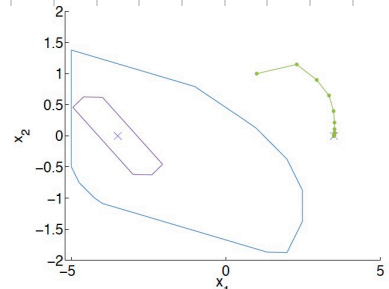
This is typically handled by running MPC on estimated states.

linear combination of states

Summary: MPC for tracking

- Set point tracking problem $\xrightarrow{\text{coordinate transformation}}$ regulation problem
- If CL system stable $\rightarrow$ set point achieved
- Proved stability using same tools as for regulation case
- Difficulties:

- Terminal set is function of target (x_s, u_s)
- changing reference can render opt. problem infeasible
- Controller will show offset in case of disturbances or unknown model error



Disturbance Rejection

To eliminate steady-state error we always used integrators up until now.

In constrained control we model the disturbance (estimation).

Then we find control inputs that use this disturbance estimate to remove offset.

Model assuming constant disturbances:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + Bd d_k \\ d_{k+1} &= d_k \\ y_k &= Cx_k + C_d d_k \end{aligned} \quad (1)$$

The choice of Bd, C_d is restricted, because the augmented model needs to be observable.

Augmented model is observable iff (A, C) observable and $\begin{bmatrix} A-I & Bd \\ C & C_d \end{bmatrix}$ has full column rank.
i.e. $\text{rank} = n_x + n_d \Rightarrow \text{max. dimension of disturbance: } n_d \leq n_y$.

Why? rewrite (1) at steady-state: $\begin{bmatrix} A-I & Bd \\ C & C_d \end{bmatrix} \begin{bmatrix} x_s \\ d_s \end{bmatrix} = \begin{bmatrix} 0 \\ y_s \end{bmatrix}$

can't estimate more disturbances than you have sensors

given y_s, d_s must be uniquely defined.

Linear State Estimation

To estimate the state and disturbance we use a Luenberger observer.

State observer for augmented model:

$$\begin{bmatrix} \hat{x}(k+1) \\ \hat{d}(k+1) \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} \hat{x}(k) \\ \hat{d}(k) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(k) + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \underbrace{(-y(k) + C\hat{x}(k) + C_d\hat{d}(k))}_{\hat{y}(k) - y(k)} \quad (2)$$

Why does this work?

Look at the error dynamics: ← using (1) & (2)

$$\begin{aligned} \begin{bmatrix} x(k+1) - \hat{x}(k+1) \\ d(k+1) - \hat{d}(k+1) \end{bmatrix} &= \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x(k) - \hat{x}(k) \\ d(k) - \hat{d}(k) \end{bmatrix} \\ &\quad - \begin{bmatrix} L_x \\ L_d \end{bmatrix} (C\hat{x}(k) + C_d\hat{d}(k) - Cx(k) - C_d d(k)) \\ &= \left(\begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \begin{bmatrix} C & C_d \end{bmatrix} \right) \begin{bmatrix} x(k) - \hat{x}(k) \\ d(k) - \hat{d}(k) \end{bmatrix} \end{aligned}$$

choose $L = \begin{bmatrix} L_x \\ L_d \end{bmatrix}$, s.t. this matrix has stable poles
 $\Rightarrow$ error will converge

Now implement the estimated states in (1) at steady-state:

- If $n_y = n_d$
- If observer asy. stable

$$\begin{bmatrix} A-I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_\infty \\ u_\infty \end{bmatrix} = \begin{bmatrix} -B_d \hat{d}_\infty \\ y_\infty - C_d \hat{d}_\infty \end{bmatrix}$$

This means with measured outputs & input we can reconstruct the state & disturbance.

Offset-free tracking

We can now integrate the above with tracking aswell, i.e. $z(k) = Hx(k) \rightarrow r$ for $k \rightarrow \infty$

New condition at steady-state:

$$\begin{aligned} x_s &= Ax_s + Bu_s + B_d \hat{d}_\infty \quad \leftarrow \text{we use our estimate } \hat{d} \\ z_s &= H(Cx_s + C_d \hat{d}_\infty) = r \end{aligned}$$

So the adapted target condition which achieves offset-free tracking:

$$\begin{bmatrix} A-I & B \\ HC & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} -B_d \hat{d} \\ r - HC_d \hat{d} \end{bmatrix} \quad (3)$$

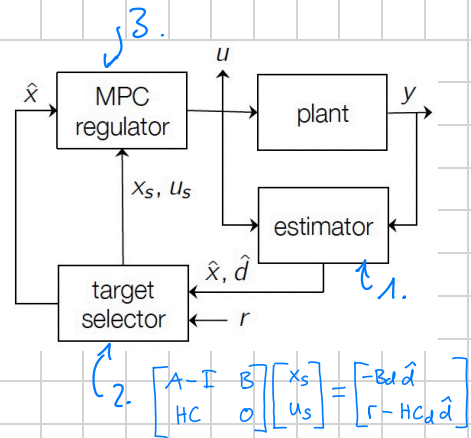
Summary: Offset-free tracking

At each sampling time:

1. Estimate $\hat{x}, \hat{d}$
2. obtain (x_s, u_s) from (3) using estimates
3. Solve MPC for tracking

This gives a theoretical result if

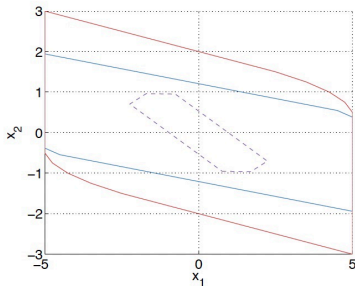
- # disturbance states = # outputs
- target steady-state problem is feasible & no constraints active at steady-state



Enlargening Feasible Set

1st approach...

- The terminal constraint reduces feasible set
- Adds state constraints to problems with only input constraints



- Blue line: Feasible set with terminal constraint
- Red line: Feasible set without terminal constraint
- Dashed line: Terminal set

Goal: MPC without terminal constraint with guaranteed stability.

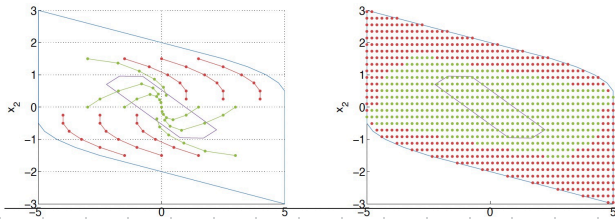
We can remove terminal constraint while maintaining stability if:

- N is large enough
- initial state is within a sufficiently small subset of feasible set

region of attraction

With this terminal state implicitly satisfies terminal constraint (without being formulated in opt. problem)

$\Rightarrow$ Solution of finite horizon MPC $\hat{=}$ infinite horizon solution

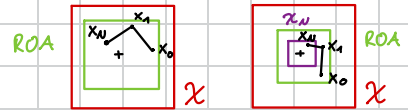


$\Rightarrow$ Controller defined in larger feasible set

$\Rightarrow$ Required horizon length N & region of attraction hard to specify

$\Rightarrow$ Terminal set would 100% guarantee stability, here it doesn't

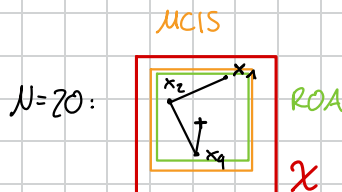
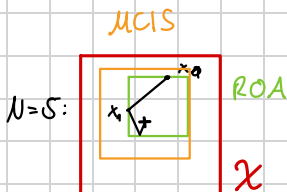
$\Rightarrow$ Region of attraction w/o terminal constraint can be larger than for MPC case



$\Rightarrow$ As N is increased: region of attraction $\rightarrow$ maximum control invariant set

largest set of starting conditions,
where there at least exists one seq. of optimal
control inputs

it can look better into the future $\nearrow$



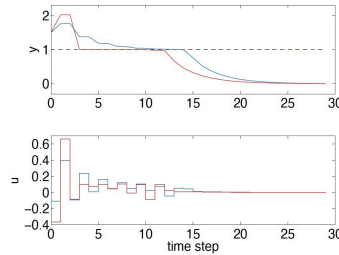
2nd way of enlarging feasible set...

Soft Constraints

- Input constraints are given by physical constraints on actuators $\Rightarrow$ hard constraints
- State/output constraints arise from practical constraints on allowed operation range $\Rightarrow$ soft constraints
- Hard constraints complicate the controller $\Rightarrow$ soften state constraints

Even when the constraints are soft, we are still trying to minimize their violation by

- Minimizing duration of violation
- Minimizing size of violation



Red: Minimum time of violation
Blue: Minimum size of violation

These goals are conflicting though! We can't minimize both simultaneously.

$\nearrow$ e.g. size of violation $\nabla \neq$ duration of violation $\nearrow$

To help us find an optimal tradeoff between size & duration of violation we can make use of **Pareto curves** (used offline)

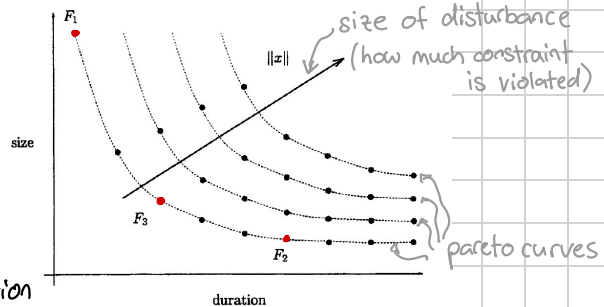
$\nwarrow$ represent tradeoff

- Best operation points lie on Pareto optimal curve.

$\Rightarrow$ points below cannot be attained

$\Rightarrow$ points above are worse

To actually find optimal point on curve engineer is responsible & depends on application



$\Rightarrow$ minimum size? Pick F_2

$\Rightarrow$ minimum duration? Pick F_1

Soft Constraints on MPC

To implement soft state constraints we use slack variables.

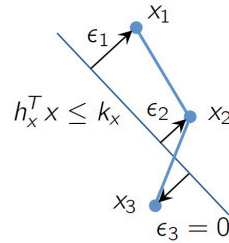
$$\min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + l_\epsilon(\epsilon_i) + x_N^T P x_N + l_\epsilon(\epsilon_N)$$

$$\text{s.t. } x_{i+1} = A x_i + B u_i$$

$$H_x x_i \leq k_x + \epsilon_i$$

$$H_u u_i \leq k_u$$

$$\epsilon_i \geq 0$$



- Slack variables ϵ_i to relax state constraints

- penalize amount of constraint violation in cost with l_ϵ

How do we choose the cost l_ϵ for soft constraints?

Let's look at the problem more broadly:

Original problem ^{hard constrained}

$$\begin{aligned} \min_z & f(z) \\ \text{subi. to } & g(z) \leq 0 \end{aligned}$$

"Softened" problem

$$\min_{z, \epsilon} f(z) + l_\epsilon(\epsilon)$$

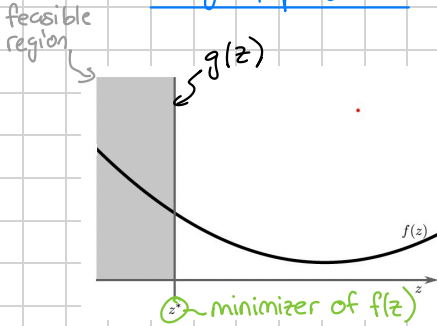
$$\begin{aligned} \text{subi. to } & g(z) \leq \epsilon \\ & \epsilon \geq 0 \end{aligned}$$

- We only want to interfere when necessary, i.e. when original problem is infeasible
- If the original problem is feasible, the relaxed problem should yield the same result

Quadratic Penalty

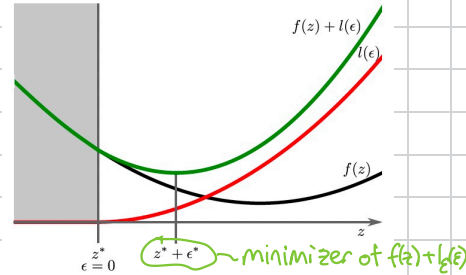
Let's start by looking at a quadratic cost for the soft constraints.

Original problem



- Constraint: $g(z) = z - z^* \leq 0$

introduce
soft constraints



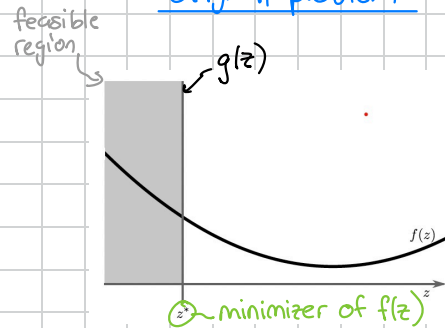
- Quadratic penalty: $l_\epsilon = s \cdot \epsilon^2$ for $\epsilon \geq 0$
0 else

This is bad! Even though problem is feasible, the solution is outside feasible set.

Linear Penalty

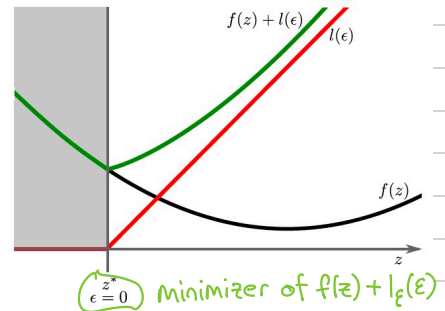
What if we try a linear penalty?

Original problem



- Constraint: $g(z) = z - z^* \leq 0$

introduce
soft constraints



- Linear penalty: $l_\epsilon(\epsilon) = v \cdot \epsilon$ for $\epsilon \geq 0$
0 else

If we choose v large enough to stop the solver from drifting into infeasible region $\Rightarrow z^*$ minimizer

Why? Because $(f(z) + l_\varepsilon(\varepsilon))' = 0$ and $f'(z) < 0$ so $l'_\varepsilon(\varepsilon) = v \stackrel{!}{>} 0$

In practice one combines linear and quadratic penalty.

to keep feasible problem
inside the constraint

to penalize large violation

If you have multiple constraints we can extend $l_\varepsilon(\varepsilon) = v \cdot \|\varepsilon\|_{1/\infty} + \varepsilon^T S \varepsilon$

Now we know how to choose the penalty,
technically can solve MPC problem:

$$\begin{aligned} \min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + l_\varepsilon(\varepsilon_i) + x_N^T P x_N + l_\varepsilon(\varepsilon_N) \\ \text{s.t. } x_{i+1} = A x_i + B u_i \\ H_x x_i \leq k_x + \varepsilon_i \\ H_u u_i \leq k_u \\ \varepsilon_i \geq 0 \end{aligned}$$

Separation of Objectives

Often times we want to separate the problem above into a two step problem

1. Minimize violation over horizon

$$\begin{aligned} \varepsilon_{\min} = \arg \min_{u, \varepsilon} \sum_{i=0}^{N-1} \varepsilon_i^T S \varepsilon_i + v^T \varepsilon_i \\ \text{s.t. } x_{i+1} = A x_i + B u_i \\ H_x x_i \leq k_x + \varepsilon_i \\ H_u u_i \leq k_u \\ \varepsilon_i \geq 0 \end{aligned}$$

2. Optimize controller performance

$$\begin{aligned} \min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + x_N^T P x_N \\ \text{s.t. } x_{i+1} = A x_i + B u_i \\ H_x x_i \leq k_x + \varepsilon_i^{\min} \\ H_u u_i \leq k_u \end{aligned}$$

⇒ Simplifies tuning, constraints satisfied if possible

⇒ Requires solution of two opt. problems

Summary

Tracking:

- Rewrite problem in delta formulation
- Setup target steady state problem to calculate (x_s, u_s)

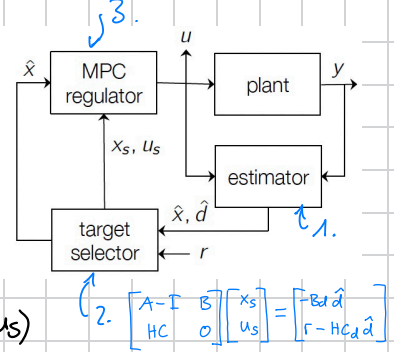
$\Rightarrow$ tracking $\hat{=}$ regulation

Disturbance Rejection:

- Augment model including disturbance
- estimate $\hat{x}$ & $\hat{d}$
- adapt target steady state problem using $\hat{d}$

Soft Constraints:

- Recover feasibility when constraints cannot be satisfied
- Introduce slack variables & choose penalty on these
- Violation durations vs size



Exercise

reminder: { tracking steady-state:
$$\begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$

LQR: $u = Kx$, $K = -(R+B^T P B)^{-1} B^T P A$

DARE: $P = Q + A^T P A - A^T P B (R+B^T P B)^{-1} B^T P A$

Consider the system

$$x(k+1) = \begin{bmatrix} 1.1 & 0.1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} u(k) \quad (1)$$

$$y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k) \quad (2)$$

with initial condition $x(0) = [-0.7, 1.4]^T$ and stage cost $l(x, u) = x^T Q x + u^T R u$ with $Q = I_2$, $R = 1$. Design an LQR controller such that $y(k) \rightarrow r$ for $k \rightarrow \infty$ and $r \in \mathbb{R}$.

1. find (x_s, u_s)

2. obtain tracking cost (with delta formulation)

3. LQR for delta formulation

solve for P :

LQR

Input: